



Proyecto POSTGIS

Municipio de Estepona Málaga

Bryan Andrés Botello Sarmiento - babotsar@upv.edu.es

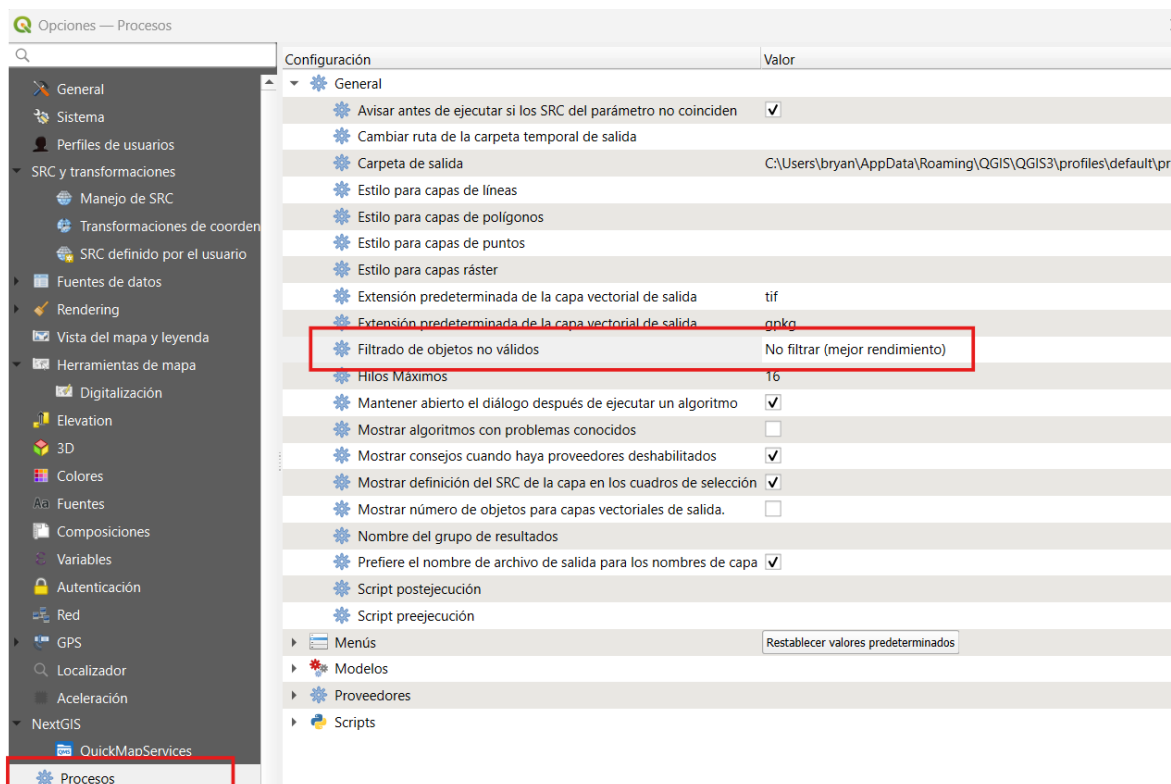
Universitat Politècnica de Valencia
Master universitario en ingeniería geomática y geoinformación

Contenido

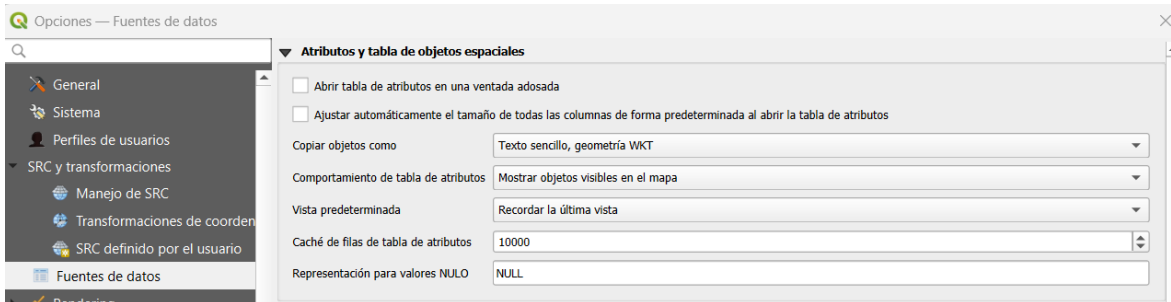
1.	Preparación de entorno QGIS	2
2.	Preparación de entorno para apoyo con inteligencia artificial de contexto	3
2.1.	Preparación de fuentes de contexto	4
2.2.	Creación de archivo de contexto y metodología de trabajo	4
3.	Descarga de datos.....	4
3.1.	Descarga de ficheros Buldings y Parcelas.	5
3.2.	Descarga de datos de Transporte.	6
3.3.	Descarga información hidrográfica	6
3.4.	Descarga información temática SIOSE	6
3.5.	Exploración inicial de los datos descargados.....	7
4.	Importación de los datos al esquema 1 (jmc1).....	10
5.	Transformaciones y validaciones para el esquema 2 (jcm2).....	11
5.1.	Termino municipal	12
5.2.	Tabla Buildings	12
5.2.1.	Candidatos	14
5.2.2.	Saneados	15
5.2.3.	Clasificados.....	16
5.2.4.	Destino	16
5.2.5.	Métricas	16
5.2.6.	Validación transaccional.....	17
5.3.	BuildingParts	18
5.4.	Cadastral Parcels:	20
5.5.	Tramos viales.....	21
5.6.	Portales y PK	23
5.7.	Hidrografía	25
5.8.	SIOSE	27
5.9.	índices espaciales y Constraint	29
5.10.	Resultados.....	31
6.	asd	33

1. Preparación de entorno QGIS

Para evaluar la calidad de las descargas de datos, es necesario evitar el filtrado de geometrías no validas.



Además de la visualización de la tabla de atributos para elementos visibles unicamente en el mapa. Para optimizar y prevenir que el programa se cuelgue.



2. Preparación de entorno para apoyo con inteligencia artificial de contexto

Para realizar este proyecto se optó por la utilización de un Agente de código de contexto como **Antigravity IDE**, la cual permite al usuario realizar consultas a un LLM que tiene a su disposición la lectura de todo el marco de trabajo, permitiendo dar respuestas más acertadas; es capaz de gestionar, modificar y leer código y hacer uso de la terminal para automatizar test.

Estos agentes regularmente pueden mal interpretar instrucciones u omitir información, por lo cual se utiliza además un sistema de control de código fuente (SCM por sus siglas en ingles) utilizando dos herramientas **Git** que se encarga localmente del control de versiones y **GitHub** que funciona como plataforma de alojamiento en la Nube para protección y sincronización de la información.

Se crea el repositorio en GitHub de nuestro proyecto en el siguiente vinculo:

https://github.com/andreesbotello/BDE_SQL.git

y Git instalado en el ordenador.

```
C:\Users\bryan>git -v  
git version 2.54.0.windows.1
```

Una vez existen ambos los conecto

```
C:\Users\bryan\Desktop\UPV\S2\1_Distribucion\BLOQUE1\Proyecto_Final>git remote -v  
origin https://github.com/andreesbotello/BDE_SQL.git (fetch)  
origin https://github.com/andreesbotello/BDE_SQL.git (push)
```

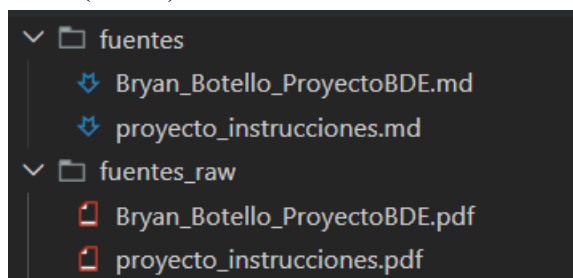
Entro al IDE y guardo el entorno de trabajo en el archivo

“Proyecto_Final.code-workspace”

Esto permite parametrizar la IDE a la lectura y gestión de los archivos únicamente de a carpeta del proyecto.

2.1. Preparación de fuentes de contexto

Primero, nos apoyamos en las fuentes del instructivo y este documento en desarrollo en formato pdf. Lo procesamos con el fichero “**convertir_fuentes.py**” con la librería `markitdown`^[1] que se ha demostrado reduce el consumo de tokens al pasar las fuentes de contexto de pdf a markdown (“.md”).



2.2. Creación de archivo de contexto y metodología de trabajo

Una vez tengamos los archivos fuente, es recomendable diseñar un archivo que permita darle instrucciones persistentes al modelo. Generalmente llamado “README.md”.

3. Descarga de datos

Antes de descargar cualquier información, se creó un archivo de configuración que personaliza el municipio de interés y la base de datos involucrada en el proyecto, llamado “**config.py**”.

```
# Configuración global del proyecto de automatización PostGIS
from datos import PROVINCIAS_MAP

# Código del municipio de estudio (5 dígitos).
CODIGO_MUNICIPIO = "29051"
NOMBRE_MUNICIPIO = "Estepona"
# Datos descriptivos y técnicos asociados al municipio
PROVINCIA = "Málaga"
PROV_MAY = "MALAGA"
SRID_PROYECTO = 25830

# Parámetros de conexión a la base de datos PostgreSQL
DB_HOST = "localhost"
DB_PORT = 5432
DB_USER = "postgres"
DB_PASSWORD = "pg"
DB_NAME = "proyecto_final"

# Nivel de influencia para el análisis de hidrografía
# Valores posibles: 'servidumbre' (10m), 'policia' (100m), 'general' (500m)
NIVEL_INFLUENCIA_HIDRO = "general"

#operaciones
PROVINCIA_DESCARGA = PROVINCIAS_MAP.get(CODIGO_PROVINCIA, PROV_MAY)
CODIGO_PROVINCIA = CODIGO_MUNICIPIO[:2]
```

Utilizando el fichero **descargas.py** se automatiza la descarga conociendo el URL de cada uno. El cual utiliza esta función sencilla para llamar las URLs de descarga.

3.1. Descarga de ficheros Buldings y Parcelas.

```
def descargar(url: str, nombre_fichero: str):
    """Descarga el archivo desde ``url`` y lo guarda en ``DESCARGAS_DIR`` con ``nombre_fichero``.
    Si el archivo ya existe, se omite la descarga.
    """
    ruta_destino = DESCARGAS_DIR / nombre_fichero
    if ruta_destino.exists():
        print(f"Archivo ya existe: {ruta_destino}")
        return ruta_destino
    print(f"Descargando {url} -> {ruta_destino}")
    try:
        urllib.request.urlretrieve(url, ruta_destino)
        print("Descarga completada.")
    except Exception as e:
        print(f"Error al descargar {url}: {e}")
    return ruta_destino
```

<https://www.catastro.hacienda.gob.es/INSPIRE/CadastralParcels/29/29051-ESTEPONA/A.ES.SDGC.CP.29051.zip>

y almacenarla apropiadamente la respuesta de la URL. Para finalmente ejecutarlo así

```
# 4.1. Catastro Parcelas Catastrales en GML ATOM
print("\n--- 4.1. Descargando Parcelas Catastrales ---")
parcelas_url = (
    f"https://www.catastro.hacienda.gob.es/INSPIRE/CadastralParcels/{CODIGO_PROVINCIA}/"
    f"{CODIGO_MUNICIPIO}-{NOMBRE_MUNICIPIO.upper().replace(' ', '')}/"
    f"A.ES.SDGC.CP.{CODIGO_MUNICIPIO}.zip"
)
descargar(parcelas_url, f"Parcelas_{CODIGO_MUNICIPIO}.zip")
```

3.2. Descarga de datos de Transporte.

Para esto se a realizado un proceso de automatización de descarga complejo, debido a que el botón de descarga de <https://centrodedescargas.cnig.es/CentroDescargas/informacion-geografica-referencia> no tiene un link directo, por el tema del licenciamiento de descarga. Sin embargo, al realizar la descarga desde la “canasta de compra”, ha permitido descargar un archivo con extensión .jnlp, evaluando su funcionamiento se identificó que trabaja bajo el lenguaje xml, que conecta con un pequeño programa de extensión .jar que trabaja con JAVA. Al inspeccionar el programa notamos que lo único que hace es leer la licencia y reenviar al centro de descargas con la licencia que me han otorgado.

URL de mi licencia:

<https://centrodedescargas.cnig.es/CentroDescargas/generarFichero.do?codLicencia=LIGW267074566>

Respuesta de la URL:

http://ftpcdd.cnig.es/PUBLICACION_CNIG_DATOS_VARIOS/red_transporte/RT_MALAGA_shp.zip|67.83|9150739

Esta información por fin nos da la URL del archivo de descarga. Una vez identificamos el descargable. Podemos automatizarlo.

```
# 1.3. Redes de Transporte (CNIG - Shapefile Provincial)
# Se descarga la red de transporte provincial correspondiente al municipio
transport_url = f"http://ftpcdd.cnig.es/PUBLICACION_CNIG_DATOS_VARIOS/red_transporte/RT_{PROVINCIA_DESCARGA}_shp.zip"
descargar(transport_url, f"RT_{PROVINCIA_DESCARGA}_shp.zip")
```

3.3. Descarga información hidrográfica

Para poder automatizar la descarga de la información de cuenca hidrográfica, se descargo toda la información comprendida en su pagina web, obteniendo las geometrías de las cuencas, se procedió a calcular la envolvente de cada cuenca para obtener un polígono al cual se le realizaron 3 buffers para determinar las áreas de influencia (10m servidumbre, 100m policía, 500m general). Así obteniendo la información del municipio y estas geometrías se puede determinar que se descargue únicamente la cuenca de interés. Respecto al vinculo de descarga, se identifico de la misma forma que datos de transporte.

http://ftpcdd.cnig.es/PUBLICACION_CNIG_DATOS_VARIOS/hidrografia/DH_V0_ES091_Ebro.ZIP

3.4.Descarga información temática SIOSE

Continuando con el mismo procedimiento, se identificó y automatizo el vinculo de descarga.

http://ftpcdd.cnig.es/SIOSE/SIOSE_2014/SIOSE_Andalucia_2014_GPKG.zip

3.5.Exploración inicial de los datos descargados

Se utiliza el archivo “**generar_metadata.py**” para obtener la información de que contine cada elemento. Codificación, numero de ficheros, sistemas de coordenadas.

Descomprimiendo y revisando la información se obtiene la siguientes tablas para el Municipio de Estepona.

	Parcelas	Edificios
Codificación	UTF-8	ISO-8859-1
Sistema de referencia (SRS)	ETRS89 / UTM zone 30N EPSG:25830	ETRS89 / UTM zone 30N EPSG:25830
Archivos internos	A.ES.SDGC.CP.29051.cadastralparcel.gml A.ES.SDGC.CP.29051.cadastralzoning.gml A.ES.SDGC.CP.MD.29051.xml	A.ES.SDGC.BU.29051.building.gml A.ES.SDGC.BU.29051.buildingpart.gml A.ES.SDGC.BU.29051.otherconstruction.gml A.ES.SDGC.BU.MD.29051.xml
Conteo de elementos	Parcelas Catastrales: 16.985 Zonificación Catastral: 1.594	Edificios: 11.758 Partes de edificios: 58.908 Otras Construcciones: 4.409

La inclusión de estos elementos estructurales en los archivos cadastralparcel.gml, building.gml y buildingpart.gml garantiza el cumplimiento de las directrices técnicas de las Directivas INSPIRE para la armonización de datos espaciales. Al integrar campos normativos como areaValue y localId para las parcelas catastrales, el uso actual (currentUse), el número de unidades de edificación (numberOfBuildingUnits) y la superficie oficial (officialArea/value) para los edificios, así como el número de plantas sobre y bajo rasante (numberOfFloorsAboveGround y numberOfFloorsBelowGround) en las partes de los edificios, los conjuntos de datos proporcionan una semántica estandarizada y una interoperabilidad transfronteriza total de acuerdo con las especificaciones de los temas de Catastro (CP) y Edificios (BU) de INSPIRE.

	Red De transporte	Red de Hidrografía
Codificación	ISO-8859-1	ISO-8859-1
Sistema de referencia (SRS)	ETRS89 EPSG:4258	ETRS89 EPSG:4258
Archivos internos	rt_pkffcc_p.shp rt_tramofc_linea.shp rt_nodoffcc_p.shp rt_estacionffcc_p.shp rt_areaffcc_s.shp	ge_pozo_p_ES060.shp ge_surgencia_p_ES060.shp hi_aguaestanc_s_ES060.shp hi_cruce_1_ES060.shp hi_cuenca_s_ES060.shp

	rt_nodotra_p.shp rt_tramo_vial.shp rt_portalpk_p.shp rt_puntoctra_p.shp rt_areactra_s.shp rt_cable_l.shp rt_nodocable_p.shp rt_areaaereo_s.shp rt_nodoaereo_p.shp rt_aerodromo_p.shp rt_conexion_a.shp rt_areamar_s.shp rt_nodomar_p.shp rt_lineamar_l.shp rt_puerto_p.shp	hi_estructuracostera_l_ES060.shp hi_laminaartificial_s_ES060.shp hi_presa_l_ES060.shp hi_presa_s_ES060.shp hi_rednodo_p_ES060.shp hi_redsecuencia_l_ES060.shp hi_redtramo_l_ES060.shp hi_subcuenca_s_ES060.shp hi_tramocurso_l_ES060.shp hi_tramocurso_s_ES060.shp hi_vado_l_ES060.shp hi_zhumeda_s_ES060.shp re_demarcacion_s_ES060.shp rm_regionmar_s_ES060.shp
Conteo de elementos	Tramos viales: 250.707 Portales y PKs: 385,381	Tramos de Cursos de Agua: 41.477

La integración de los elementos estructurales contenidos en el conjunto de datos provincial en formato Shapefile garantiza la conformidad con las especificaciones técnicas de la Directiva INSPIRE para el tema de Redes de Transporte (TN). Al incorporar campos normativos clave dentro de los tramos viales (rt_tramo_vial), tales como los identificadores únicos (id_tramo, id_vial), la tipología vial (clased), el odónimo (nombre) y el tipo de firme (firmed), junto con la indexación posicional de portales y puntos kilométricos (rt_portalpk), definidos por sus identificadores estructurales (id_porpk) y su numeración oficial (numero), el conjunto de datos proporciona una semántica estandarizada bajo el Sistema de Referencia Espacial ETRS89 (EPSG:4258). Esto asegura una interoperabilidad transfronteriza total y una continuidad topológica óptima para la gestión de las redes de movilidad urbana e interurbana.

Mientras que para Red de hidrografía, la inclusión de la información geográfica correspondiente a la demarcación de las Cuencas Mediterráneas Andaluzas asegura el estricto cumplimiento de las directrices técnicas de las Directivas INSPIRE dentro del tema de Hidrografía (HY). A través del tratamiento de los tramos de cursos de agua (hi_tramocurso_l_ES060), definidos en el sistema geodésico de referencia ETRS89 (EPSG:4258), se integran atributos alfanuméricos fundamentales como el identificador único del curso (id_curso), el hidrónimo oficial (nombre) y el régimen hidrológico o tipología del flujo (tipo_curso). La consistencia de estos campos garantiza la armonización y codificación normalizada de los datos espaciales, permitiendo el análisis hidrológico integrado y la explotación interoperable de la red de drenaje superficial a escala europea.

	Limites Base	SIOSE
Codificación	UTF-8	UTF-8
Sistema de	ETRS89	ETRS89 / UTM zone 30N

referencia (SRS)	EPSG:4258	EPSG:25830
Archivos internos	SHP_ETRS89/II_autonomicas_inspire_peninbal_etr89/II_autonomicas_inspire_peninbal_etr89.shp SHP_ETRS89/II_municipales_inspire_peninbal_etr89/II_municipales_inspire_peninbal_etr89.shp SHP_ETRS89/II_provinciales_inspire_peninbal_etr89/II_provinciales_inspire_peninbal_etr89.shp SHP_ETRS89/recintos_autonomicas_inspire_peninbal_etr89/recintos_autonomicas_inspire_peninbal_etr89.shp SHP_ETRS89/recintos_municipales_inspire_peninbal_etr89/recintos_municipales_inspire_peninbal_etr89.shp SHP_ETRS89/recintos_provinciales_inspire_peninbal_etr89/recintos_provinciales_inspire_peninbal_etr89.shp SHP_REGCAN95/II_autonomicas_inspire_canarias_regcan95/II_autonomicas_inspire_canarias_regcan95.shp SHP_REGCAN95/II_municipales_inspire_canarias_regcan95/II_municipales_inspire_canarias_regcan95.shp SHP_REGCAN95/II_provinciales_inspire_canarias_regcan95/II_provinciales_inspire_canarias_regcan95.shp SHP_REGCAN95/recintos_autonomicas_inspire_canarias_regcan95/recintos_autonomicas_inspire_canarias_regcan95.shp SHP_REGCAN95/recintos_municipales_inspire_canarias_regcan95/recintos_municipales_inspire_canarias_regcan95.shp SHP_REGCAN95/recintos_provinciales_inspire_canarias_regcan95/recintos_provinciales_inspire_canarias_regcan95.shp Zona neutral entre España y Marruecos en Ceuta/zonaneutral Marruecos-	SIOSE_Andalucia_2014.gpkg tablas relacionales: CODIIGE HILUX

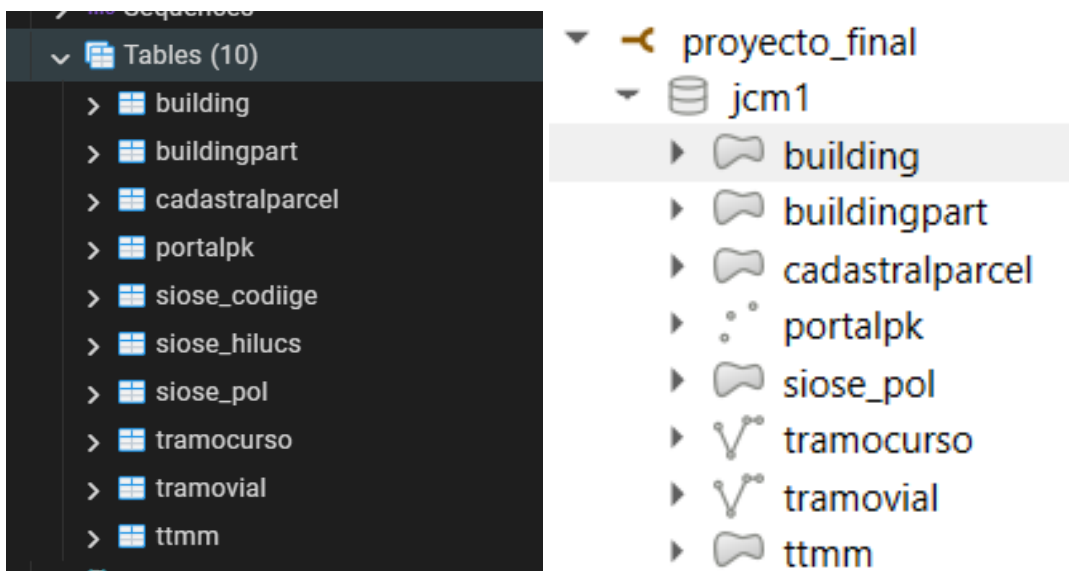
	Ceuta.shp Zona neutral entre España y Marruecos en Melilla/Zona Neutral Marruecos- Melilla.shp	
Conteo de elementos	Parcelas Catastrales: 16.985 Zonificación Catastral: 1.594	Polígonos de Ocupación: 766.791

Con respecto a las líneas base, la integración de la capa de recintos municipales a nivel nacional asegura la conformidad con las especificaciones técnicas de la Directiva INSPIRE para el tema de Unidades Administrativas (AU). Al incorporar los campos normativos obligatorios, tales como el código nacional de la unidad administrativa (NATCODE) — utilizado para aislar geográficamente la entidad correspondiente a Estepona—, el nombre oficial del término municipal (NAMEUNIT) y el identificador unívoco para el catálogo europeo (INSPIREID), el conjunto de datos proporciona una semántica estandarizada bajo el Sistema de Referencia Espacial ETRS89 (EPSG:4258). Esto garantiza la interoperabilidad transfronteriza total, facilitando la cohesión y el análisis de los límites jurisdiccionales oficiales en la infraestructura de datos espaciales común.

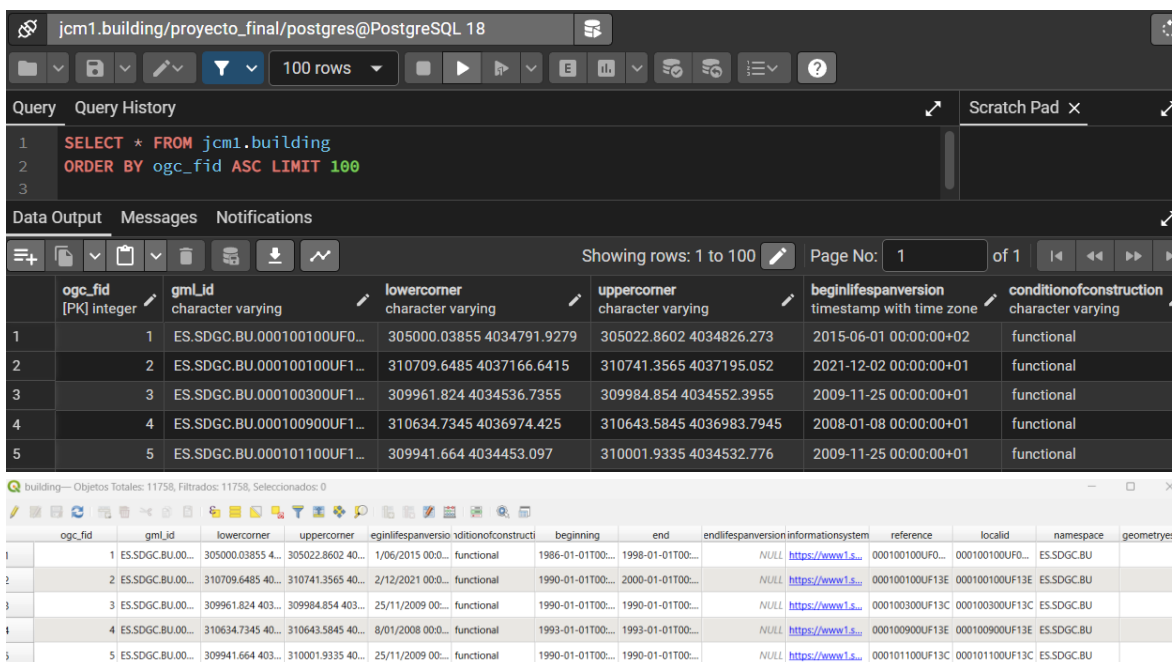
Mientras que para el SIOSE, la inclusión de la información cartográfica y alfanumérica de ocupación del suelo a escala autonómica garantiza el estricto cumplimiento de las directrices técnicas de las Directivas INSPIRE dentro del tema de Usos del Suelo (LU). Mediante la estructuración relacional de los polígonos de ocupación (T_POLIGONOS) y sus tablas auxiliares de cobertura (t_siose_codiige) y uso (t_siose_hilucs), definidos en el sistema proyectado ETRS89 / UTM zone 30N (EPSG:25830), se asocian de forma íntegra atributos clave como el identificador único de la geometría (ID_POLYGON), el descriptor jerárquico de cobertura (CODIIGE) y la clasificación de uso estandarizada a nivel europeo (HILUCS). Esta consistencia de campos y descripciones normalizadas asegura una interoperabilidad transfronteriza total para la monitorización ambiental, la planificación territorial y el análisis dinámico del territorio.

4. Importación de los datos al esquema 1 (jmc1)

Para aprovechar la librería GDAL, se va a hacer uso del Python que viene con la descarga de QGIS. Y haciendo uso de OSGeo4W Shell ejecutaremos los scripts. De tal manera que se genera el fichero que automatiza la importación de la información de los comprimidos denominado “**importar_jcm1.py**”. Adicional, se generan consultas SQL conectadas a través de psycopg2 para generar la importación con el procesamiento en GDAL, consultas en .sql y compilado en python.



Revisión en PgAdmin de la existencia de los datos.



5. Transformaciones y validaciones para el esquema 2 (jcm2)

La transformación de datos se llevó a cabo en el mismo sistema híbrido entre el compilado python y las consultas sql. Donde se implementó el primer grupo de consultas “procesar_jcm2.sql” el cual realizaba una limpieza inicial de las tablas, creaba tablas de cero como por ejemplo la de building. Para las 8 tablas definitivas de este esquema.

```
-- 2.2. Edificios (building)
CREATE TABLE jcm2.building (
  gid serial PRIMARY KEY,
  gml_id varchar,
  current_use_in varchar,
  currentuse varchar,
  numberofbuildingunits integer,
  value integer,
  geom geometry(MultiPolygon, 25830)
);
```

5.1. Termino municipal

Se crea primero el termino municipal con un buffer de 500 metros para luego filtrar el espacio de trabajo basado en este criterio.

```
-- 3.1. Insertar el Municipio de Estudio (Estepona)
-- SRID de origine 4258
INSERT INTO jcm2.ttmm (inspireid, natcode, nameunit, geom)
SELECT inspireid, natcode, nameunit, ST_Multi(ST_Force2D(ST_MakeValid(ST_Transform(geom, {{SRID_PROYECTO}}))))
FROM jcm1.ttmm
WHERE (natcode = '3417' || SUBSTRING('{{CODIGO_MUNICIPIO}}', 1, 2) || '{{CODIGO_MUNICIPIO}}'
      OR natcode LIKE '%' || '{{CODIGO_MUNICIPIO}}')
      AND geom IS NOT NULL;

-- Crear el índice espacial permanente e inmediato en jcm2.ttmm
CREATE INDEX jcm2_ttmm_geom_idx ON jcm2.ttmm USING gist(geom);
```

Criterios: Se optó por utilizar ST_MakeValid ya que esta operación primero realiza ST_IsValid. Por lo que hacer la operación, IsValid y luego MakeValid haría que IsValid pase dos veces. Y ST_Multi para que coincida con la columna de destino. El filtrado sigue la regla que se uso para la descarga de datos donde el código completo es 34172929051.

Junto con su índice espacial para optimizar las consultas futuras sobre las capas siguientes.

5.2. Tabla Buildings

En una versión inicial se realizaba esta operación

```
-- 3.2. Insertar Edificios (dentro del buffer de 500m)
-- Se corrigen valores negativos, se mapea el uso de dominio y se filtran geometrías < 0.5m2 preventivamente.
INSERT INTO jcm2.building (gml_id, current_use_in, currentuse, numberofbuildingunits, value, geom)
SELECT b.gml_id,
       b.currentuse, -- Preservar el valor original indexado para uso interno
       CASE
         WHEN b.currentuse = '1_residential' THEN 'residential'
         WHEN b.currentuse = '2_agriculture' THEN 'agriculture'
         WHEN b.currentuse = '3_industrial' THEN 'industrial'
         WHEN b.currentuse = '4_2_retail' THEN 'commerceAndServices'
         WHEN b.currentuse = '4_3_publicServices' THEN 'publicServices'
         WHEN b.currentuse = '4_1_office' THEN 'office'
         ELSE NULL
       END,
       GREATEST(0, b.numberofbuildingunits),
       GREATEST(0, b.value),
       ST_Multi(ST_Force2D(ST_MakeValid(ST_Transform(b.geom, 25830))))
FROM jcm1.building b, jcm2.ttmm m
WHERE ST_DWithin(b.geom, m.geom, 500)
      AND b.geom IS NOT NULL
      AND ST_Area(ST_Force2D(ST_MakeValid(ST_Transform(b.geom, 25830)))) >= 0.5;
```

El cual presentaba bastantes inconsistencias, con baja trazabilidad y alta demanda de cómputo. Se solicitó al LLM de Antigravity mejorar dicho código dándole parámetros claros sobre como realizarlo una vez. Obteniendo una serie de consultas procedurales tipo ETL (Extraer, Transformar, Cargar), el cual sigue el principio de paso a paso para la extracción, validación, transformación y carga de datos, ideal para fuentes heterogéneas como este proyecto. Obteniendo.

```
-- Paso A: Filtro municipal (ST_DWithin reprojectando al vuelo si difieren)
INSERT INTO jcm2.stage_building (gml_id, current_use_in, numberofbuildingunits, value, geom_raw, srid_original, es_valida)
SELECT b.gml_id, b.currentuse, b.numberofbuildingunits, b.value, b.geom, ST_SRID(b.geom), ST_IsValid(b.geom)
FROM jcm1.building b, jcm2.ttmm m
WHERE b.geom IS NOT NULL AND ST_DWithin(
  CASE WHEN ST_SRID(b.geom) = {{SRID_PROYECTO}} THEN b.geom ELSE ST_Transform(b.geom, {{SRID_PROYECTO}}) END,
  m.geom,
  500
);

-- Paso B: Reproyección y Corrección de Geometría (Solo a no válidas)
UPDATE jcm2.stage_building SET geom_temp = ST_Transform(geom_raw, {{SRID_PROYECTO}});
UPDATE jcm2.stage_building SET geom_temp = ST_MakeValid(geom_temp) WHERE NOT es_valida_original;
UPDATE jcm2.stage_building
SET es_valida_post_correccion = ST_IsValid(geom_temp),
    vacia_post_correccion = ST_IsEmpty(geom_temp);

-- Paso C: Mapeo de Atributos Semánticos e INSPIRE
UPDATE jcm2.stage_building
SET currentuse = CASE
  WHEN current_use_in = '1_residential' THEN 'residential'
  WHEN current_use_in = '2_agriculture' THEN 'agriculture'
  WHEN current_use_in = '3_industrial' THEN 'industrial'
  WHEN current_use_in = '4_2_retail' THEN 'commerceAndServices'
  WHEN current_use_in = '4_3_publicServices' THEN 'publicServices'
  WHEN current_use_in = '4_1_office' THEN 'office'
  ELSE NULL
END,
numberofbuildingunits = GREATEST(0, numberofbuildingunits),
value = GREATEST(0, value);
```

```
-- Paso D: Conversión a 2D (Force2D) y Verificación de daños
UPDATE jcm2.stage_building SET geom_temp = ST_Multi(ST_Force2D(geom_temp));
UPDATE jcm2.stage_building
SET descartada_conversion = true
WHERE NOT ST_IsValid(geom_temp) OR ST_IsEmpty(geom_temp);

-- Paso E: Filtro de Escala/Microgeometrias
UPDATE jcm2.stage_building
SET descartada_escal = true
WHERE ST_Area(geom_temp) < 0.5;

-- Paso F: Registro de Métricas de Calidad
INSERT INTO jcm2.log_calidad_geometrias (tabla, total_origen_buffer, originales_validas, originales_invalidas, reparadas)
SELECT
    'building',
    COUNT(*),
    COUNT(CASE WHEN es_valida_original THEN 1 END),
    COUNT(CASE WHEN NOT es_valida_original THEN 1 END),
    COUNT(CASE WHEN NOT es_valida_original AND es_valida_post_correccion AND NOT vacia_post_correccion THEN 1 END),
    COUNT(CASE WHEN (NOT es_valida_original AND NOT es_valida_post_correccion) OR vacia_post_correccion THEN 1 END),
    COUNT(CASE WHEN descartada_conversion THEN 1 END),
    COUNT(CASE WHEN descartada_escal THEN 1 END)
FROM jcm2.stage_building;

-- Paso G: Carga Definitiva
INSERT INTO jcm2.building (gml_id, current_use_in, currentuse, numberofbuildingunits, value, geom)
SELECT gml_id, current_use_in, currentuse, numberofbuildingunits, value, geom_temp
FROM jcm2.stage_building
WHERE es_valida_post_correccion
    AND NOT vacia_post_correccion
    AND NOT descartada_conversion
    AND NOT descartada_escal;

-- Paso H: Limpieza
DROP TABLE IF EXISTS jcm2.stage_building;
```

La cual no generó un aumento considerable del cómputo, mejoró la claridad y permitió ir llevando registro en `jcm2.log_calidad_geometrias` sobre la calidad de los datos, y cada transformación que se obtuvo. Sin embargo. Tenía aun posibilidades de mejora. Por lo que luego se le solicitó a la IA de Claude Sonnet 4.6. un análisis de dicha información y oportunidades de mejora. Generando una filosófica CTE (Common Table Expression) que mejoraba la fase de transformación del esquema ETL, la cual recomendaba evitar tablas temporales, prevenir efectos de botella y optimización de consultas. Obteniendo el siguiente esquema de procesamiento.

5.2.1. Candidatos

```
WITH candidatos AS (  
  SELECT  
    b.gml_id,  
    b.currentuse AS current_use_in,  
    b.numberofbuildingunits AS units_raw,  
    b.value AS value_raw,  
    b.geom AS geom_raw,  
    ST_SRID(b.geom) AS srid_raw,  
    ST_Transform(b.geom, {{SRID_PROYECTO}}) AS geom_proj  
  FROM jcm1.building b  
  CROSS JOIN (SELECT ST_Union(geom) AS geom FROM jcm2.ttmm) m  
  WHERE b.geom IS NOT NULL  
    AND ST_DWithin(  
      ST_Transform(b.geom, {{SRID_PROYECTO}}),  
      m.geom,  
      500  
    )  
),
```

Obtenemos la información filtrada por las columnas de interés de jcm1.building, luego se hace un CROSS JOIN sobre el termino municipal, en caso de que tenga más de una geometría, que no es el caso, pero la idea es dejarlo universal, con municipios que pueden tener exclaves. Con este polígono unido se hace el filtrado por ST_DWithin que en el curso repetidas veces se demostró ser más optimo que ST_Distance.

5.2.2. Saneados

```
saneados AS (  
  SELECT  
    c.*,  
    ST_IsValid(c.geom_proj) AS valida_en_proj,  
    l.geom_final,  
    ST_IsValid(l.geom_final) AS valida_final,  
    ST_IsEmpty(l.geom_final) AS vacia_final  
  FROM candidatos c  
  CROSS JOIN LATERAL (  
    SELECT ST_Multi(ST_Force2D(CASE WHEN ST_IsValid(c.geom_proj) THEN c.geom_proj ELSE ST_MakeValid(c.geom_proj) END)) AS geom_final  
  ) l  
),
```

Una vez se construye los primeros elementos de candidatos, estos son datos son pasado por un CASE WHEN ST_IsValid para que el ST_MakeValid solo se realice sobre geometrías que lo requiere, las que pasan son ajustadas al modelo necesario con ST_Force2D (aplanado), ST_Multi para que coincida con la geometría de destino y en la misma consulta calcular el área para pasar por cada fila una sola vez.

5.2.3. Clasificados

```
clasificados AS (  
  SELECT  
    srid_raw,  
    gml_id,  
    current_use_in,  
    CASE current_use_in  
      WHEN '1_residential' THEN 'residential'  
      WHEN '2_agriculture' THEN 'agriculture'  
      WHEN '3_industrial' THEN 'industrial'  
      WHEN '4_2_retail' THEN 'commerceAndServices'  
      WHEN '4_3_publicServices' THEN 'publicServices'  
      WHEN '4_1_office' THEN 'office'  
      WHEN '5_educational' THEN 'educational'  
      WHEN '6_health' THEN 'health'  
      WHEN '7_recreational' THEN 'recreational'  
      WHEN '8_other' THEN 'other'  
      WHEN '9_ancillary' THEN 'ancillary'  
      ELSE NULL  
    END  
    COALESCE(GREATEST(0, units_raw), 0) AS currentuse,  
    COALESCE(GREATEST(0, value_raw), 0) AS numberofbuildingunits,  
    AS value,  
    geom_raw,  
    geom_proj,  
    geom_final,  
    valida_en_proj,  
    valida_final,  
    vacia_final,  
    (NOT valida_en_proj AND (NOT valida_final OR vacia_final)) AS es_corrupta,  
    (valida_final AND NOT vacia_final AND (geom_final IS NULL OR NOT ST_IsValid(geom_final) OR ST_IsEmpty(geom_final))) AS es_rota_conversion,  
    (valida_final AND NOT vacia_final AND geom_final IS NOT NULL AND ST_IsValid(geom_final) AND NOT ST_IsEmpty(geom_final) AND ST_Area(geom_final) < 0.5) AS es_filtrada_escal,  
    (valida_final AND NOT vacia_final AND geom_final IS NOT NULL AND ST_IsValid(geom_final) AND NOT ST_IsEmpty(geom_final) AND ST_Area(geom_final) >= 0.5) AS es_apta  
  FROM saneados  
)
```

Una vez superan la reparación geométrica, se realiza el computo de operaciones como ajustar a la nomenclatura INSPIRE. Y realiza la limpieza de datos nulos en los campos de Numberofbuildingunits y value. Los cuales a partir de la función COALESCE hace que cualquier dato negativo o null pase forzosamente a ser 0. Y finalmente los “flags” de estado donde, para dar mayor trazabilidad se obtiene si la geometría es corrupta (desde el origen), se ha roto en conversión (no soportó un colapso vertical del forcé 2D), de escala menor a 0.5m2 o totalmente apta.

5.2.4. Destino

```
insert_destino AS (  
  INSERT INTO jcm2.building (gml_id, current_use_in, currentuse, numberofbuildingunits, value, geom)  
  SELECT gml_id, current_use_in, currentuse, numberofbuildingunits, value, geom_final  
  FROM clasificados  
  WHERE es_apta  
  RETURNING gid  
)  
  
insert_auditoria AS (  
  INSERT INTO jcm2.log_detalle_calidad (tabla, gml_id, srid_original, es_valida_original, valida_post_corr, vacia_post_corr, motivo_descarte, geom_original)  
  SELECT  
    'building', gml_id, srid_raw, valida_en_proj, valida_final, vacia_final,  
    CASE  
      WHEN es_corrupta THEN 'corrupta'  
      WHEN es_rota_conversion THEN 'rota_conversion'  
      WHEN es_filtrada_escal THEN 'escala_micro'  
    END,  
    ST_Force2D(geom_proj)  
  FROM clasificados  
  WHERE NOT es_apta OR NOT valida_en_proj  
)
```

Similar a la lógica de FME se construye un enrutamiento donde los aptos pasan a jcm2.building y los errores detectados en la clasificación pasan al log de detalle para servir de información del proceso.

5.2.5. Métricas

Otro gran acierto de la separación de la consulta en múltiples partes fue la obtención de métricas más detalladas con la siguiente línea. Únicamente calcula e inserta los datos a log de calidad.

```
metricas AS (  
  SELECT  
    COUNT(*) AS total_origen_buffer,  
    COUNT(*) FILTER (WHERE valida_en_proj) AS originales_validas,  
    COUNT(*) FILTER (WHERE NOT valida_en_proj) AS originales_invalidas,  
    COUNT(*) FILTER (WHERE NOT valida_en_proj AND es_apta) AS reparadas_exito,  
    COUNT(*) FILTER (WHERE es_corrupta) AS corruptas_descartadas,  
    COUNT(*) FILTER (WHERE es_rota_conversion) AS filtradas_conversion_2d,  
    COUNT(*) FILTER (WHERE es_filtrada_escalas) AS filtradas_escalas,  
    (SELECT COUNT(*) FROM insert_destino) AS insertadas_destino,  
    COALESCE(MAX(srid_raw), 0) AS srid_raw  
  FROM clasificados  
)  
INSERT INTO jcm2.log_calidad_geometrias (  
  tabla, ts_proceso, srid_origen, srid_proyecto, total_origen_buffer,  
  originales_validas, originales_invalidas, reparadas_exito, corruptas_descartadas,  
  filtradas_conversion_2d, filtradas_escalas, insertadas_destino, notas  
)  
SELECT  
  'building', now(), srid_raw, {{SRID_PROYECTO}}, total_origen_buffer,  
  originales_validas, originales_invalidas, reparadas_exito, corruptas_descartadas,  
  filtradas_conversion_2d, filtradas_escalas, insertadas_destino,  
  CASE  
    WHEN EXISTS (  
      SELECT 1 FROM clasificados  
      WHERE current_use_in IS NOT NULL AND currentuse IS NULL  
    )  
    THEN 'ADVERTENCIA: existen valores de current_use_in sin mapeo INSPIRE.'  
    ELSE NULL  
  END  
FROM metricas;
```

5.2.6. Validación transaccional

Por ultimo se crea una función anónima con plpgsql que calcula la suma de edificios descartados más los insertados dio igual al numero de edificios filtrados al inicio.

```
-- Validación de consistencia lógica para building  
DO $$  
DECLARE  
  r jcm2.log_calidad_geometrias%ROWTYPE;  
  suma_categorias integer;  
BEGIN  
  SELECT * INTO r FROM jcm2.log_calidad_geometrias WHERE tabla = 'building';  
  suma_categorias := COALESCE(r.corruptas_descartadas, 0) + COALESCE(r.filtradas_conversion_2d, 0) + COALESCE(r.filtradas_escalas, 0) + COALESCE(r.insertadas_destino, 0);  
  IF r.total_origen_buffer IS DISTINCT FROM suma_categorias THEN  
    RAISE EXCEPTION 'LOG INCONSISTENTE para building: total-% != suma-%', r.total_origen_buffer, suma_categorias;  
  END IF;  
  RAISE NOTICE 'building - % procesados + % insertados', r.total_origen_buffer, r.insertadas_destino;  
END;  
$$;  
ANALYZE jcm2.building;
```

Al tratarse de una función anónima poco vista en el curso voy a desglosar más detalladamente.

DO \$\$: hace una función anónima que no se almacena la ejecuta inmediatamente junto con el delimitador del símbolos dolar.

DECLARE: define las variables que se van a usar en la función

Las variables utilizadas fueron

- rjcm2.log_calidad_geometrias%ROWTYPE: aquí “r” es el nombre y todo lo demás es el tipo de variable, que es muy robusta, que es un registro con la misma naturaleza que las columnas de log.calidad_geoemtrias.
- Suma_categorias: es un numero entero

BEGIN: da el fin de DECLARE y el inicio a la función.

SELECT * INTO r FROM jcm2.log_calidad_geometrias WHERE tabla = 'building': almacena este resultado del SELECT en la variable r.

Luego se define la variable suma_categorias

suma_categorias := COALESCE(r.corruptas_descartadas, 0) +
COALESCE(r.filtradas_conversion_2d, 0) + COALESCE(r.filtradas_escalas, 0) +
COALESCE(r.insertadas_destino, 0): aquí se obtiene el numero total de registros en todas las categorías de las “flags”.

IF r.total_origen_buffer IS DISTINCT FROM suma_categorias THEN

RAISE EXCEPTION 'LOG INCONSISTENTE para building: total=% != suma=%',
r.total_origen_buffer, suma_categorias; aquí simplemente se hace el condicional si los datos filtrados inicialmente no corresponden a la suma de todas las flags. Emitir

'LOG INCONSISTENTE para building: total=% != suma=%', r.total_origen_buffer,
suma_categorias;

De lo contrario emitir.

RAISE NOTICE 'building · % procesados → % insertados', r.total_origen_buffer,
r.insertadas_destino;

Y por ultimo como recomendación de Claude.

ANALYZE jcm2.building; que garantiza que el motor de postgres es conciente de la arquitectura de jcm2 y en consultas futuras lo llama de forma óptima.

Este principio operativo se aplica para todas las demás tablas.

5.3. BuildingParts

```
-- 3.3. Procesamiento de Partes de Edificios (buildingpart)
-----
DELETE FROM jcm2.log_calidad_geometrias WHERE tabla = 'buildingpart';

WITH candidatos AS (
  SELECT
    bp.gml_id,
    bp.numberoffloorsaboveground AS floors_above_raw,
    bp.numberoffloorsbelowground AS floors_below_raw,
    bp.geom AS geom_raw,
    ST_SRID(bp.geom) AS srid_raw,
    ST_Transform(bp.geom, {{SRID_PROYECTO}}) AS geom_proj
  FROM jcm1.buildingpart bp
  CROSS JOIN (SELECT ST_Union(geom) AS geom FROM jcm2.ttmm) m
  WHERE bp.geom IS NOT NULL
  AND ST_DWithin(
    ST_Transform(bp.geom, {{SRID_PROYECTO}}),
    m.geom,
    500
  )
),
```

```
saneados AS (
  SELECT
    c.*,
    ST_IsValid(c.geom_proj) AS valida_en_proj,
    l.geom_final,
    ST_IsValid(l.geom_final) AS valida_final,
    ST_IsEmpty(l.geom_final) AS vacia_final
  FROM candidatos c
  CROSS JOIN LATERAL (
    SELECT ST_Multi(ST_Force2D(CASE WHEN ST_IsValid(c.geom_proj) THEN c.geom_proj ELSE
ST_MakeValid(c.geom_proj) END)) AS geom_final
  ) l
),
clasificados AS (
  SELECT
    srid_raw,
    gml_id,
    COALESCE(GREATEST(0, floors_above_raw), 0) AS numberoffloorsaboveground,
    COALESCE(GREATEST(0, floors_below_raw), 0) AS numberoffloorsbelowground,
    geom_raw,
    geom_proj,
    geom_final,
    valida_en_proj,
    valida_final,
    vacia_final,
    (NOT valida_en_proj AND (NOT valida_final OR vacia_final)) AS es_corrupta,
    (valida_final AND NOT vacia_final AND (geom_final IS NULL OR NOT ST_IsValid(geom_final) OR
ST_IsEmpty(geom_final))) AS es_rota_conversion,
    (valida_final AND NOT vacia_final AND geom_final IS NOT NULL AND ST_IsValid(geom_final) AND
NOT ST_IsEmpty(geom_final) AND ST_Area(geom_final) < 0.5) AS es_filtrada_escal,
    (valida_final AND NOT vacia_final AND geom_final IS NOT NULL AND ST_IsValid(geom_final) AND
NOT ST_IsEmpty(geom_final) AND ST_Area(geom_final) >= 0.5) AS es_apt
  FROM saneados
),
insert_destino AS (
  INSERT INTO jcm2.buildingpart (gml_id, numberoffloorsaboveground, numberoffloorsbelowground,
geom)
  SELECT gml_id, numberoffloorsaboveground, numberoffloorsbelowground, geom_final
  FROM clasificados
  WHERE es_apt
  RETURNING gid
),
insert_auditoria AS (
  INSERT INTO jcm2.log_detalle_calidad (tabla, gml_id, srid_original, es_valida_original,
valida_post_corr, vacia_post_corr, motivo_descarte, geom_original)
  SELECT
    'buildingpart', gml_id, srid_raw, valida_en_proj, valida_final, vacia_final,
    CASE
      WHEN es_corrupta THEN 'corrupta'
      WHEN es_rota_conversion THEN 'rota_conversion'
      WHEN es_filtrada_escal THEN 'escala_micro'
    END,
    ST_Force2D(geom_proj)
  FROM clasificados
  WHERE NOT es_apt OR NOT valida_en_proj
),
metricas AS (
  SELECT
    COUNT(*) AS total_origen_buffer,
    COUNT(*) FILTER (WHERE valida_en_proj) AS originales_validas,
    COUNT(*) FILTER (WHERE NOT valida_en_proj) AS originales_invalidas,
    COUNT(*) FILTER (WHERE NOT valida_en_proj AND es_apt) AS reparadas_exito,
    COUNT(*) FILTER (WHERE es_corrupta) AS corruptas_descartadas,
    COUNT(*) FILTER (WHERE es_rota_conversion) AS filtradas_conversion_2d,
    COUNT(*) FILTER (WHERE es_filtrada_escal) AS filtradas_escal,
    (SELECT COUNT(*) FROM insert_destino) AS insertadas_destino,
    COALESCE(MAX(srid_raw), 0) AS srid_raw
  FROM clasificados
)
```

```
INSERT INTO jcm2.log_calidad_geometrias (
    tabla, ts_proceso, srid_origen, srid_proyecto, total_origen_buffer,
    originales_validas, originales_invalidas, reparadas_exito, corruptas_descartadas,
    filtradas_conversion_2d, filtradas_escalas, insertadas_destino, notas
)
SELECT
    'buildingpart', now(), srid_raw, {{SRID_PROYECTO}}, total_origen_buffer,
    originales_validas, originales_invalidas, reparadas_exito, corruptas_descartadas,
    filtradas_conversion_2d, filtradas_escalas, insertadas_destino, NULL
FROM metricas;

-- Validación de consistencia lógica para buildingpart
DO $$
DECLARE
    r jcm2.log_calidad_geometrias%ROWTYPE;
    suma_categorias integer;
BEGIN
    SELECT * INTO r FROM jcm2.log_calidad_geometrias WHERE tabla = 'buildingpart';
    suma_categorias := COALESCE(r.corruptas_descartadas, 0) + COALESCE(r.filtradas_conversion_2d, 0)
+ COALESCE(r.filtradas_escalas, 0) + COALESCE(r.insertadas_destino, 0);
    IF r.total_origen_buffer IS DISTINCT FROM suma_categorias THEN
        RAISE EXCEPTION 'LOG INCONSISTENTE para buildingpart: total=% != suma=%',
r.total_origen_buffer, suma_categorias;
    END IF;
    RAISE NOTICE 'buildingpart · % procesados → % insertados', r.total_origen_buffer,
r.insertadas_destino;
END;
$$;
ANALYZE jcm2.buildingpart;
```

5.4.Cadastral Parcels:

```
-- 3.4. Procesamiento de Parcelas Catastrales (cadastralparcel)
-----
DELETE FROM jcm2.log_calidad_geometrias WHERE tabla = 'cadastralparcel';

WITH candidatos AS (
    SELECT
        cp.gml_id,
        cp.areavalue,
        cp.localid,
        cp.geom AS geom_raw,
        ST_SRID(cp.geom) AS srid_raw,
        ST_Transform(cp.geom, {{SRID_PROYECTO}}) AS geom_proj
    FROM jcm1.cadastralparcel cp
    CROSS JOIN (SELECT ST_Union(geom) AS geom FROM jcm2.tmm) m
    WHERE cp.geom IS NOT NULL
        AND ST_DWithin(
            ST_Transform(cp.geom, {{SRID_PROYECTO}}),
            m.geom,
            500
        )
),
saneados AS (
    SELECT
        c.*,
        ST_IsValid(c.geom_proj) AS valida_en_proj,
        l.geom_final,
        ST_IsValid(l.geom_final) AS valida_final,
        ST_IsEmpty(l.geom_final) AS vacia_final
    FROM candidatos c
    CROSS JOIN LATERAL (
        SELECT ST_Multi(ST_Force2D(CASE WHEN ST_IsValid(c.geom_proj) THEN c.geom_proj ELSE
ST_MakeValid(c.geom_proj) END)) AS geom_final
    ) l
),
clasificados AS (
```

```
SELECT
    srid_raw,
    gml_id,
    areavalue,
    localid,
    geom_raw,
    geom_proj,
    geom_final,
    valida_en_proj,
    valida_final,
    vacia_final,
    (NOT valida_en_proj AND (NOT valida_final OR vacia_final)) AS es_corrupta,
    (valida_final AND NOT vacia_final AND (geom_final IS NULL OR NOT ST_IsValid(geom_final) OR
    ST_IsEmpty(geom_final))) AS es_rota_conversion,
    (valida_final AND NOT vacia_final AND geom_final IS NOT NULL AND ST_IsValid(geom_final) AND
    NOT ST_IsEmpty(geom_final) AND ST_Area(geom_final) < 0.5) AS es_filtrada_escal,
    (valida_final AND NOT vacia_final AND geom_final IS NOT NULL AND ST_IsValid(geom_final) AND
    NOT ST_IsEmpty(geom_final) AND ST_Area(geom_final) >= 0.5) AS es_apt
FROM saneados
),
insert_destino AS (
    INSERT INTO jcm2.cadastralparcel (gml_id, areavalue, localid, geom)
    SELECT gml_id, areavalue, localid, geom_final
    FROM clasificados
    WHERE es_apt
    RETURNING gid
),
insert_auditoria AS (
    INSERT INTO jcm2.log_detalle_calidad (tabla, gml_id, srid_origen, es_valida_original,
    valida_post_corr, vacia_post_corr, motivo_descarte, geom_original)
    SELECT
        'cadastralparcel', gml_id, srid_raw, valida_en_proj, valida_final, vacia_final,
        CASE
            WHEN es_corrupta THEN 'corrupta'
            WHEN es_rota_conversion THEN 'rota_conversion'
            WHEN es_filtrada_escal THEN 'escal_micro'
        END,
        ST_Force2D(geom_proj)
    FROM clasificados
    WHERE NOT es_apt OR NOT valida_en_proj
),
metricas AS (
    SELECT
        COUNT(*) AS total_origen_buffer,
        COUNT(*) FILTER (WHERE valida_en_proj) AS originales_validas,
        COUNT(*) FILTER (WHERE NOT valida_en_proj) AS originales_invalidas,
        COUNT(*) FILTER (WHERE NOT valida_en_proj AND es_apt) AS reparadas_exito,
        COUNT(*) FILTER (WHERE es_corrupta) AS corruptas_descartadas,
        COUNT(*) FILTER (WHERE es_rota_conversion) AS filtradas_conversion_2d,
        COUNT(*) FILTER (WHERE es_filtrada_escal) AS filtradas_escal,
        (SELECT COUNT(*) FROM insert_destino) AS insertadas_destino,
        COALESCE(MAX(srid_raw), 0) AS srid_raw
    FROM clasificados
)
INSERT INTO jcm2.log_calidad_geometrias (
    tabla, ts_proceso, srid_origen, srid_proyecto, total_origen_buffer,
    originales_validas, originales_invalidas, reparadas_exito, corruptas_descartadas,
    filtradas_conversion_2d, filtradas_escal, insertadas_destino, notas
)
SELECT
    'cadastralparcel', now(), srid_raw, {{SRID_PROYECTO}}, total_origen_buffer,
    originales_validas, originales_invalidas, reparadas_exito, corruptas_descartadas,
    filtradas_conversion_2d, filtradas_escal, insertadas_destino, NULL
FROM metricas;
```

5.5. Tramos viales

Para tramos viales la única diferencia importante con el sistema para polígonos es sustituir el calculo de área por longitud. Y se ha decidido retirar la condición y el constraint para retirar polígonos de menos de 0.5m, ya que la perdida de tramos de transición pequeños podría desconectar las redes viales.

```
-- 3.5. Procesamiento de Tramos Viales (tramovial)
-----
DELETE FROM jcm2.log_calidad_geometrias WHERE tabla = 'tramovial';

WITH candidatos AS (
  SELECT
    tv.id_tramo,
    tv.id_vial,
    tv.clased,
    tv.nombre,
    tv.firmed,
    tv.geom AS geom_raw,
    ST_SRID(tv.geom) AS srid_raw,
    ST_Transform(tv.geom, {{SRID_PROYECTO}}) AS geom_proj
  FROM jcm1.tramovial tv
  CROSS JOIN (SELECT ST_Union(geom) AS geom FROM jcm2.ttmm) m
  WHERE tv.geom IS NOT NULL
    AND ST_DWithin(
      ST_Transform(tv.geom, {{SRID_PROYECTO}}),
      m.geom,
      500
    )
),
saneados AS (
  SELECT
    c.*,
    ST_IsValid(c.geom_proj) AS valida_en_proj,
    l.geom_final,
    ST_IsValid(l.geom_final) AS valida_final,
    ST_IsEmpty(l.geom_final) AS vacia_final
  FROM candidatos c
  CROSS JOIN LATERAL (
    SELECT ST_Multi(ST_Force2D(CASE WHEN ST_IsValid(c.geom_proj) THEN c.geom_proj
ELSE ST_MakeValid(c.geom_proj) END)) AS geom_final
  ) l
),
clasificados AS (
  SELECT
    srid_raw,
    id_tramo,
    id_vial,
    clased,
    nombre,
    firmed,
    geom_raw,
    geom_proj,
    geom_final,
    valida_en_proj,
    valida_final,
    vacia_final,
    (NOT valida_en_proj AND (NOT valida_final OR vacia_final)) AS es_corrupta,
    -- Non-simple or invalid/empty lines count under es_rota_conversion
    (valida_final AND NOT vacia_final AND (geom_final IS NULL OR NOT
ST_IsValid(geom_final) OR ST_IsEmpty(geom_final) OR NOT ST_IsSimple(geom_final))) AS
es_rota_conversion,
```

```
(valida_final AND NOT vacia_final AND geom_final IS NOT NULL AND
ST_IsValid(geom_final) AND NOT ST_IsEmpty(geom_final) AND ST_IsSimple(geom_final) AND
ST_Length(geom_final) < 0.5) AS es_filtrada_escalas,
(valida_final AND NOT vacia_final AND geom_final IS NOT NULL AND
ST_IsValid(geom_final) AND NOT ST_IsEmpty(geom_final) AND ST_IsSimple(geom_final) AND
ST_Length(geom_final) >= 0.5) AS es_aptas
FROM saneados
),
insert_destino AS (
INSERT INTO jcm2.tramovial (id_tramo, id_vial, clased, nombre, firmado, geom)
SELECT id_tramo, id_vial, clased, nombre, firmado, geom_final
FROM clasificados
WHERE es_aptas
RETURNING gid
),
insert_auditoria AS (
INSERT INTO jcm2.log_detalle_calidad (tabla, gml_id, srid_original,
es_valida_original, valida_post_corr, vacia_post_corr, motivo_descarte, geom_original)
SELECT
'tramovial', id_tramo, srid_raw, valida_en_proj, valida_final, vacia_final,
CASE
WHEN es_corrupta THEN 'corrupta'
WHEN es_rota_conversion THEN 'rota_conversion'
WHEN es_filtrada_escalas THEN 'escalas_micro'
END,
ST_Force2D(geom_proj)
FROM clasificados
WHERE NOT es_aptas OR NOT valida_en_proj
),
metricas AS (
SELECT
COUNT(*) AS total_origen_buffer,
COUNT(*) FILTER (WHERE valida_en_proj) AS originales_validas,
COUNT(*) FILTER (WHERE NOT valida_en_proj) AS originales_invalidas,
COUNT(*) FILTER (WHERE NOT valida_en_proj AND es_aptas) AS reparadas_exito,
COUNT(*) FILTER (WHERE es_corrupta) AS corruptas_descartadas,
COUNT(*) FILTER (WHERE es_rota_conversion) AS filtradas_conversion_2d,
COUNT(*) FILTER (WHERE es_filtrada_escalas) AS filtradas_escalas,
(SELECT COUNT(*) FROM insert_destino) AS insertadas_destino,
COALESCE(MAX(srid_raw), 0) AS srid_raw
FROM clasificados
)
INSERT INTO jcm2.log_calidad_geometrias (
tabla, ts_proceso, srid_origen, srid_proyecto, total_origen_buffer,
originales_validas, originales_invalidas, reparadas_exito, corruptas_descartadas,
filtradas_conversion_2d, filtradas_escalas, insertadas_destino, notas
)
SELECT
'tramovial', now(), srid_raw, {{SRID_PROYECTO}}, total_origen_buffer,
originales_validas, originales_invalidas, reparadas_exito, corruptas_descartadas,
filtradas_conversion_2d, filtradas_escalas, insertadas_destino, NULL
FROM metricas;
```

5.6. Portales y PK

```
-- 3.6. Procesamiento de Portales y PKs (portalpk)
-----
DELETE FROM jcm2.log_calidad_geometrias WHERE tabla = 'portalpk';

WITH candidatos AS (
SELECT
```

```
pk.id_tramo,
pk.id_vial,
pk.id_porpk,
pk.numero,
pk.geom AS geom_raw,
ST_SRID(pk.geom) AS srid_raw,
ST_Transform(pk.geom, {{SRID_PROYECTO}}) AS geom_proj
FROM jcm1.portalpk pk
CROSS JOIN (SELECT ST_Union(geom) AS geom FROM jcm2.ttmm) m
WHERE pk.geom IS NOT NULL
AND ST_DWithin(
    ST_Transform(pk.geom, {{SRID_PROYECTO}}),
    m.geom,
    500
),
saneados AS (
    SELECT
        c.*,
        ST_IsValid(c.geom_proj) AS valida_en_proj,
        l.geom_final,
        ST_IsValid(l.geom_final) AS valida_final,
        ST_IsEmpty(l.geom_final) AS vacia_final
    FROM candidatos c
    CROSS JOIN LATERAL (
        SELECT ST_Multi(ST_Force2D(CASE WHEN ST_IsValid(c.geom_proj) THEN c.geom_proj ELSE
ST_MakeValid(c.geom_proj) END)) AS geom_final
    ) l
),
clasificados AS (
    SELECT
        srid_raw,
        id_porpk AS gml_id, -- identificador único
        id_tramo,
        id_vial,
        numero,
        geom_raw,
        geom_proj,
        geom_final,
        valida_en_proj,
        valida_final,
        vacia_final,
        (NOT valida_en_proj AND (NOT valida_final OR vacia_final)) AS es_corrupta,
        (valida_final AND NOT vacia_final AND (geom_final IS NULL OR NOT ST_IsValid(geom_final) OR
ST_IsEmpty(geom_final))) AS es_rota_conversion,
        FALSE AS es_filtrada_escalas,
        (valida_final AND NOT vacia_final AND geom_final IS NOT NULL AND ST_IsValid(geom_final) AND
NOT ST_IsEmpty(geom_final)) AS es_aptas
    FROM saneados
),
insert_destino AS (
    INSERT INTO jcm2.portalpk (id_tramo, id_vial, id_porpk, numero, geom)
    SELECT id_tramo, id_vial, gml_id, numero, geom_final
    FROM clasificados
    WHERE es_aptas
    RETURNING gid
),
insert_auditoria AS (
    INSERT INTO jcm2.log_detalle_calidad (tabla, gml_id, srid_original, es_valida_original,
valida_post_corr, vacia_post_corr, motivo_descarte, geom_original)
    SELECT
        'portalpk', gml_id, srid_raw, valida_en_proj, valida_final, vacia_final,
        CASE
            WHEN es_corrupta THEN 'corrupta'
            WHEN es_rota_conversion THEN 'rota_conversion'
            WHEN es_filtrada_escalas THEN 'escalas_micro'
        END,
        ST_Force2D(geom_proj)
    FROM clasificados
```

```
WHERE NOT es_apt OR NOT valida_en_proj
),
metricas AS (
  SELECT
    COUNT(*) AS total_origen_buffer,
    COUNT(*) FILTER (WHERE valida_en_proj) AS originales_validas,
    COUNT(*) FILTER (WHERE NOT valida_en_proj) AS originales_invalidas,
    COUNT(*) FILTER (WHERE NOT valida_en_proj AND es_apt) AS reparadas_exito,
    COUNT(*) FILTER (WHERE es_corrupta) AS corruptas_descartadas,
    COUNT(*) FILTER (WHERE es_rota_conversion) AS filtradas_conversion_2d,
    0 AS filtradas_escalas,
    (SELECT COUNT(*) FROM insert_destino) AS insertadas_destino,
    COALESCE(MAX(srid_raw), 0) AS srid_raw
  FROM clasificados
)
INSERT INTO jcm2.log_calidad_geometrias (
  tabla, ts_proceso, srid_origen, srid_proyecto, total_origen_buffer,
  originales_validas, originales_invalidas, reparadas_exito, corruptas_descartadas,
  filtradas_conversion_2d, filtradas_escalas, insertadas_destino, notas
)
SELECT
  'portalpk', now(), srid_raw, {{SRID_PROYECTO}}, total_origen_buffer,
  originales_validas, originales_invalidas, reparadas_exito, corruptas_descartadas,
  filtradas_conversion_2d, filtradas_escalas, insertadas_destino, NULL
FROM metricas;

-- Validación de consistencia lógica para portalpk
DO $$
DECLARE
  r jcm2.log_calidad_geometrias%ROWTYPE;
  suma_categorias integer;
BEGIN
  SELECT * INTO r FROM jcm2.log_calidad_geometrias WHERE tabla = 'portalpk';
  suma_categorias := COALESCE(r.corruptas_descartadas, 0) + COALESCE(r.filtradas_conversion_2d, 0)
+ COALESCE(r.filtradas_escalas, 0) + COALESCE(r.insertadas_destino, 0);
  IF r.total_origen_buffer IS DISTINCT FROM suma_categorias THEN
    RAISE EXCEPTION 'LOG INCONSISTENTE para portalpk: total=% != suma=%', r.total_origen_buffer,
    suma_categorias;
  END IF;
  RAISE NOTICE 'portalpk · % procesados → % insertados', r.total_origen_buffer,
  r.insertadas_destino;
END;
$$;
ANALYZE jcm2.portalpk;
```

5.7. Hidrografía

```
-- 3.7. Procesamiento de Red de Hidrografía (tramocurso)
-----
DELETE FROM jcm2.log_calidad_geometrias WHERE tabla = 'tramocurso';

WITH candidatos AS (
  SELECT
    tc.id_curso,
    tc.nombre,
    tc.tipo_curso,
    tc.geom AS geom_raw,
    ST_SRID(tc.geom) AS srid_raw,
    ST_Transform(tc.geom, {{SRID_PROYECTO}}) AS geom_proj
  FROM jcm1.tramocurso tc
  CROSS JOIN (SELECT ST_Union(geom) AS geom FROM jcm2.ttmm) m
  WHERE tc.geom IS NOT NULL
    AND ST_DWithin(
      ST_Transform(tc.geom, {{SRID_PROYECTO}}),
      m.geom,
      500
    )
),
saneados AS (
```

```
SELECT
  c.*,
  ST_IsValid(c.geom_proj) AS valida_en_proj,
  l.geom_final,
  ST_IsValid(l.geom_final) AS valida_final,
  ST_IsEmpty(l.geom_final) AS vacia_final
FROM candidatos c
CROSS JOIN LATERAL (
  SELECT ST_Multi(ST_Force2D(CASE WHEN ST_IsValid(c.geom_proj) THEN c.geom_proj ELSE
ST_MakeValid(c.geom_proj) END)) AS geom_final
) l
),
clasificados AS (
  SELECT
    srid_raw,
    id_curso,
    nombre,
    tipo_curso,
    geom_raw,
    geom_proj,
    geom_final,
    valida_en_proj,
    valida_final,
    vacia_final,
    (NOT valida_en_proj AND (NOT valida_final OR vacia_final)) AS es_corrupta,
    (valida_final AND NOT vacia_final AND (geom_final IS NULL OR NOT ST_IsValid(geom_final) OR
ST_IsEmpty(geom_final))) AS es_rota_conversion,
    FALSE
    AS es_filtrada_escalas,
    (valida_final AND NOT vacia_final AND geom_final IS NOT NULL AND ST_IsValid(geom_final) AND
NOT ST_IsEmpty(geom_final)) AS es_aptas
  FROM saneados
),
insert_destino AS (
  INSERT INTO jcm2.tramocurso (id_curso, nombre, tipo_curso, geom)
  SELECT gml_id, nombre, tipo_curso, geom_final
  FROM clasificados
  WHERE es_aptas
  RETURNING gid
),
insert_auditoria AS (
  INSERT INTO jcm2.log_detalle_calidad (tabla, gml_id, srid_original, es_valida_original,
valida_post_corr, vacia_post_corr, motivo_descarte, geom_original)
  SELECT
    'tramocurso', gml_id, srid_raw, valida_en_proj, valida_final, vacia_final,
    CASE
      WHEN es_corrupta THEN 'corrupta'
      WHEN es_rota_conversion THEN 'rota_conversion'
      WHEN es_filtrada_escalas THEN 'escalas_micro'
    END,
    ST_Force2D(geom_proj)
  FROM clasificados
  WHERE NOT es_aptas OR NOT valida_en_proj
),
metricas AS (
  SELECT
    COUNT(*) AS total_origen_buffer,
    COUNT(*) FILTER (WHERE valida_en_proj) AS originales_validas,
    COUNT(*) FILTER (WHERE NOT valida_en_proj) AS originales_invalidas,
    COUNT(*) FILTER (WHERE NOT valida_en_proj AND es_aptas) AS reparadas_exito,
    COUNT(*) FILTER (WHERE es_corrupta) AS corruptas_descartadas,
    COUNT(*) FILTER (WHERE es_rota_conversion) AS filtradas_conversion_2d,
    COUNT(*) FILTER (WHERE es_filtrada_escalas) AS filtradas_escalas,
    (SELECT COUNT(*) FROM insert_destino) AS insertadas_destino,
    COALESCE(MAX(srid_raw), 0) AS srid_raw
  FROM clasificados
)
INSERT INTO jcm2.log_calidad_geometrias (
  tabla, ts_proceso, srid_origen, srid_proyecto, total_origen_buffer,
```

```
originales_validas, originales_invalidas, reparadas_exito, corruptas_descartadas,
filtradas_conversion_2d, filtradas_escalas, insertadas_destino, notas
)
SELECT
  'tramocurso', now(), srid_raw, {{SRID_PROYECTO}}, total_origen_buffer,
  originales_validas, originales_invalidas, reparadas_exito, corruptas_descartadas,
  filtradas_conversion_2d, filtradas_escalas, insertadas_destino, NULL
FROM metricas;

-- Validación de consistencia lógica para tramocurso
DO $$
DECLARE
  r jcm2.log_calidad_geometrias%ROWTYPE;
  suma_categorias integer;
BEGIN
  SELECT * INTO r FROM jcm2.log_calidad_geometrias WHERE tabla = 'tramocurso';
  suma_categorias := COALESCE(r.corruptas_descartadas, 0) + COALESCE(r.filtradas_conversion_2d, 0)
+ COALESCE(r.filtradas_escalas, 0) + COALESCE(r.insertadas_destino, 0);
  IF r.total_origen_buffer IS DISTINCT FROM suma_categorias THEN
    RAISE EXCEPTION 'LOG INCONSISTENTE para tramocurso: total=% != suma=%',
r.total_origen_buffer, suma_categorias;
  END IF;
  RAISE NOTICE 'tramocurso · % procesados → % insertados', r.total_origen_buffer,
r.insertadas_destino;
END;
$$;
ANALYZE jcm2.tramocurso;
```

5.8. SIOSE

```
-- 3.8. Procesamiento de SIOSE Polígonos (siose_pol)
-----
DELETE FROM jcm2.log_calidad_geometrias WHERE tabla = 'siose_pol';

WITH candidatos AS (
  SELECT
    s.id_polygon,
    s.codiige,
    s.hilucs,
    s.geom AS geom_raw,
    ST_SRID(s.geom) AS srid_raw,
    ST_Transform(s.geom, {{SRID_PROYECTO}}) AS geom_proj
  FROM jcm1.siose_pol s
  CROSS JOIN (SELECT ST_Union(geom) AS geom FROM jcm2.ttmm) m
  WHERE s.geom IS NOT NULL
  AND ST_DWithin(
    ST_Transform(s.geom, {{SRID_PROYECTO}}),
    m.geom,
    500
  )
),
saneados AS (
  SELECT
    c.*,
    ST_IsValid(c.geom_proj) AS valida_en_proj,
    l.geom_final,
    ST_IsValid(l.geom_final) AS valida_final,
    ST_IsEmpty(l.geom_final) AS vacia_final
  FROM candidatos c
  CROSS JOIN LATERAL (
    SELECT ST_Multi(ST_Force2D(CASE WHEN ST_IsValid(c.geom_proj) THEN c.geom_proj
ELSE ST_MakeValid(c.geom_proj) END)) AS geom_final
  ) l
),
clasificados AS (
```

```
SELECT
    srid_raw,
    id_polygon                                AS gml_id, -- identificador
único
    codiige,
    hilucs,
    geom_raw,
    geom_proj,
    geom_final,
    valida_en_proj,
    valida_final,
    vacia_final,
    (codiige NOT IN (SELECT codiige FROM jcm1.siose_codiige)
     OR hilucs NOT IN (SELECT hilucs FROM jcm1.siose_hilucs)) AS es_incoherente_ref,
    (((NOT valida_en_proj AND (NOT valida_final OR vacia_final))
     OR (codiige NOT IN (SELECT codiige FROM jcm1.siose_codiige)
        OR hilucs NOT IN (SELECT hilucs FROM jcm1.siose_hilucs)))) AS es_corrupta,
    (valida_final AND NOT vacia_final AND NOT (codiige NOT IN (SELECT codiige FROM
jcm1.siose_codiige) OR hilucs NOT IN (SELECT hilucs FROM jcm1.siose_hilucs)) AND
    (geom_final IS NULL OR NOT ST_IsValid(geom_final) OR ST_IsEmpty(geom_final))) AS
es_rota_conversion,
    (valida_final AND NOT vacia_final AND NOT (codiige NOT IN (SELECT codiige FROM
jcm1.siose_codiige) OR hilucs NOT IN (SELECT hilucs FROM jcm1.siose_hilucs)) AND
    geom_final IS NOT NULL AND ST_IsValid(geom_final) AND NOT ST_IsEmpty(geom_final) AND
    ST_Area(geom_final) < 0.5) AS es_filtrada_escalas,
    (valida_final AND NOT vacia_final AND NOT (codiige NOT IN (SELECT codiige FROM
jcm1.siose_codiige) OR hilucs NOT IN (SELECT hilucs FROM jcm1.siose_hilucs)) AND
    geom_final IS NOT NULL AND ST_IsValid(geom_final) AND NOT ST_IsEmpty(geom_final) AND
    ST_Area(geom_final) >= 0.5) AS es_apta
    FROM saneados
),
insert_destino AS (
    INSERT INTO jcm2.siose_pol (id_polygon, codiige, hilucs, geom)
    SELECT gml_id, codiige, hilucs, geom_final
    FROM clasificados
    WHERE es_apta
    RETURNING gid
),
insert_auditoria AS (
    INSERT INTO jcm2.log_detalle_calidad (tabla, gml_id, srid_original,
es_valida_original, valida_post_corr, vacia_post_corr, motivo_descarte, geom_original)
    SELECT
        'siose_pol', gml_id, srid_raw, valida_en_proj, valida_final, vacia_final,
    CASE
        WHEN es_corrupta THEN 'corrupta'
        WHEN es_rota_conversion THEN 'rota_conversion'
        WHEN es_filtrada_escalas THEN 'escalas_micro'
    END,
    ST_Force2D(geom_proj)
    FROM clasificados
    WHERE NOT es_apta OR NOT valida_en_proj
),
metricas AS (
    SELECT
        COUNT(*)                                AS total_origen_buffer,
        COUNT(*) FILTER (WHERE valida_en_proj)   AS originales_validas,
        COUNT(*) FILTER (WHERE NOT valida_en_proj) AS originales_invalidas,
        COUNT(*) FILTER (WHERE NOT valida_en_proj AND es_apta) AS reparadas_exito,
        COUNT(*) FILTER (WHERE es_corrupta)       AS corruptas_descartadas,
        COUNT(*) FILTER (WHERE es_rota_conversion) AS filtradas_conversion_2d,
        COUNT(*) FILTER (WHERE es_filtrada_escalas) AS filtradas_escalas,
```

```
(SELECT COUNT(*) FROM insert_destino)          AS insertadas_destino,
COALESCE(MAX(srid_raw), 0)                      AS srid_raw
FROM clasificados
)
INSERT INTO jcm2.log_calidad_geometrias (
    tabla, ts_proceso, srid_origen, srid_proyecto, total_origen_buffer,
    originales_validas, originales_invalidas, reparadas_exito, corruptas_descartadas,
    filtradas_conversion_2d, filtradas_escalas, insertadas_destino, notas
)
SELECT
    'siose_pol', now(), srid_raw, {{SRID_PROYECTO}}, total_origen_buffer,
    originales_validas, originales_invalidas, reparadas_exito, corruptas_descartadas,
    filtradas_conversion_2d, filtradas_escalas, insertadas_destino, NULL
FROM metricas;

-- Validación de consistencia lógica para siose_pol
DO $$
DECLARE
    r jcm2.log_calidad_geometrias%ROWTYPE;
    suma_categorias integer;
BEGIN
    SELECT * INTO r FROM jcm2.log_calidad_geometrias WHERE tabla = 'siose_pol';
    suma_categorias := COALESCE(r.corruptas_descartadas, 0) +
COALESCE(r.filtradas_conversion_2d, 0) + COALESCE(r.filtradas_escalas, 0) +
COALESCE(r.insertadas_destino, 0);
    IF r.total_origen_buffer IS DISTINCT FROM suma_categorias THEN
        RAISE EXCEPTION 'LOG INCONSISTENTE para siose_pol: total=% != suma=%',
r.total_origen_buffer, suma_categorias;
    END IF;
    RAISE NOTICE 'siose_pol · % procesados → % insertados', r.total_origen_buffer,
r.insertadas_destino;
END;
$$;
ANALYZE jcm2.siose_pol;
```

5.9. índices espaciales y Constraint

Finalmente teniendo las 8 tablas depuradas en jcm2 procedo a crear los índices espaciales para mejorar la optimización de las consultas a esta base de datos y las vistas de jcm3. Además de las políticas restrictivas que se usaron para depurar se convierten en reglas para los nuevos elementos ingresados.

```
-- 5. CREACIÓN DE ÍNDICES DEFINITIVOS
-- =====
-- Índices espaciales definitivos (bulk load ya ejecutado)
CREATE INDEX jcm2_building_geom_idx ON jcm2.building USING gist(geom);
CREATE INDEX jcm2_buildingpart_geom_idx ON jcm2.buildingpart USING gist(geom);
CREATE INDEX jcm2_cadastralparcel_geom_idx ON jcm2.cadastralparcel USING gist(geom);
CREATE INDEX jcm2_tramovial_geom_idx ON jcm2.tramovial USING gist(geom);
CREATE INDEX jcm2_portalpk_geom_idx ON jcm2.portalpk USING gist(geom);
CREATE INDEX jcm2_tramocurso_geom_idx ON jcm2.tramocurso USING gist(geom);
CREATE INDEX jcm2_siose_pol_geom_idx ON jcm2.siose_pol USING gist(geom);

-- Índice espacial de la tabla de auditoría detallada
CREATE INDEX jcm2_log_detalle_calidad_geom_idx ON jcm2.log_detalle_calidad USING
gist(geom_original);

-- Índices de atributos
CREATE INDEX jcm2_building_currentuse_idx ON jcm2.building (currentuse);
CREATE INDEX jcm2_building_current_use_in_idx ON jcm2.building (current_use_in);

-- Índice expresional para optimización de JOINs de volumen (Q8.4)
```

```
CREATE INDEX jcm2_buildingpart_gml_id_prefix_idx ON jcm2.buildingpart (LEFT(gml_id, 25));

-- Claves primarias en tablas auxiliares alfanuméricas
ALTER TABLE jcm2.siose_codiige ADD CONSTRAINT pk_siose_codiige PRIMARY KEY (codiige);
ALTER TABLE jcm2.siose_hilucs ADD CONSTRAINT pk_siose_hilucs PRIMARY KEY (hilucs);

-- Ejecutar ANALYZE en todas las tablas para actualizar estadísticas de índices
ANALYZE jcm2.ttmm;
ANALYZE jcm2.building;
ANALYZE jcm2.buildingpart;
ANALYZE jcm2.cadastralparcel;
ANALYZE jcm2.tramovial;
ANALYZE jcm2.portalpk;
ANALYZE jcm2.tramocurso;
ANALYZE jcm2.siose_pol;
ANALYZE jcm2.log_detalle_calidad;

-- 6. ADICIÓN DE RESTRICCIONES (CONSTRAINTS) SEMÁNTICAS Y GEOMÉTRICAS
-- =====
-- Nota: Uso de NOT VALID seguido de VALIDATE CONSTRAINT para optimización de bloqueos.

-- 6.1. Restricciones de Geometría Válida (ST_IsValid)
ALTER TABLE jcm2.ttmm ADD CONSTRAINT chk_ttmm_geom_valid CHECK (ST_IsValid(geom)) NOT
VALID;
ALTER TABLE jcm2.ttmm VALIDATE CONSTRAINT chk_ttmm_geom_valid;

ALTER TABLE jcm2.building ADD CONSTRAINT chk_building_geom_valid CHECK (ST_IsValid(geom))
NOT VALID;
ALTER TABLE jcm2.building VALIDATE CONSTRAINT chk_building_geom_valid;

ALTER TABLE jcm2.buildingpart ADD CONSTRAINT chk_buildingpart_geom_valid CHECK
(ST_IsValid(geom)) NOT VALID;
ALTER TABLE jcm2.buildingpart VALIDATE CONSTRAINT chk_buildingpart_geom_valid;

ALTER TABLE jcm2.cadastralparcel ADD CONSTRAINT chk_cadastralparcel_geom_valid CHECK
(ST_IsValid(geom)) NOT VALID;
ALTER TABLE jcm2.cadastralparcel VALIDATE CONSTRAINT chk_cadastralparcel_geom_valid;

ALTER TABLE jcm2.tramovial ADD CONSTRAINT chk_tramovial_geom_valid CHECK
(ST_IsValid(geom)) NOT VALID;
ALTER TABLE jcm2.tramovial VALIDATE CONSTRAINT chk_tramovial_geom_valid;

ALTER TABLE jcm2.tramocurso ADD CONSTRAINT chk_tramocurso_geom_valid CHECK
(ST_IsValid(geom)) NOT VALID;
ALTER TABLE jcm2.tramocurso VALIDATE CONSTRAINT chk_tramocurso_geom_valid;

ALTER TABLE jcm2.siose_pol ADD CONSTRAINT chk_siose_pol_geom_valid CHECK
(ST_IsValid(geom)) NOT VALID;
ALTER TABLE jcm2.siose_pol VALIDATE CONSTRAINT chk_siose_pol_geom_valid;

-- 6.2. Restricción de Elementos de Red Lineales Simples (ST_IsSimple)
ALTER TABLE jcm2.tramovial ADD CONSTRAINT chk_tramovial_geom_simple CHECK
(ST_IsSimple(geom)) NOT VALID;
ALTER TABLE jcm2.tramovial VALIDATE CONSTRAINT chk_tramovial_geom_simple;

-- 6.3. Restricciones de Dimensiones Mínimas Admisibles (Escala 1:5000)
ALTER TABLE jcm2.building ADD CONSTRAINT chk_building_geom_area CHECK (ST_Area(geom) >=
0.5) NOT VALID;
ALTER TABLE jcm2.building VALIDATE CONSTRAINT chk_building_geom_area;
```

```

ALTER TABLE jcm2.buildingpart ADD CONSTRAINT chk_buildingpart_geom_area CHECK
(ST_Area(geom) >= 0.5) NOT VALID;
ALTER TABLE jcm2.buildingpart VALIDATE CONSTRAINT chk_buildingpart_geom_area;

ALTER TABLE jcm2.cadastralparcel ADD CONSTRAINT chk_cadastralparcel_geom_area CHECK
(ST_Area(geom) >= 0.5) NOT VALID;
ALTER TABLE jcm2.cadastralparcel VALIDATE CONSTRAINT chk_cadastralparcel_geom_area;

ALTER TABLE jcm2.siose_pol ADD CONSTRAINT chk_siose_pol_geom_area CHECK (ST_Area(geom) >=
0.5) NOT VALID;
ALTER TABLE jcm2.siose_pol VALIDATE CONSTRAINT chk_siose_pol_geom_area;
-- Restricciones de longitud mínima eliminadas para preservar conectividad en redes
lineales (tramovial y tramocurso)

-- 6.4. Restricciones de Campos Alfanuméricos Positivos
ALTER TABLE jcm2.building ADD CONSTRAINT chk_building_units CHECK (numberofbuildingunits
>= 0) NOT VALID;
ALTER TABLE jcm2.building VALIDATE CONSTRAINT chk_building_units;

ALTER TABLE jcm2.building ADD CONSTRAINT chk_building_value CHECK (value >= 0) NOT VALID;
ALTER TABLE jcm2.building VALIDATE CONSTRAINT chk_building_value;

ALTER TABLE jcm2.buildingpart ADD CONSTRAINT chk_buildingpart_floors_up CHECK
(numberoffloorsaboveground >= 0) NOT VALID;
ALTER TABLE jcm2.buildingpart VALIDATE CONSTRAINT chk_buildingpart_floors_up;

ALTER TABLE jcm2.buildingpart ADD CONSTRAINT chk_buildingpart_floors_down CHECK
(numberoffloorsbelowground >= 0) NOT VALID;
ALTER TABLE jcm2.buildingpart VALIDATE CONSTRAINT chk_buildingpart_floors_down;

-- 6.5. Restricción de Dominio Acotado para currentuse (INSPIRE)
ALTER TABLE jcm2.building ADD CONSTRAINT chk_building_currentuse CHECK (
    currentuse IN (
        'residential', 'agriculture', 'industrial', 'commerceAndServices',
        'publicServices', 'office', 'educational', 'health',
        'recreational', 'other', 'ancillary'
    ) OR currentuse IS NULL
) NOT VALID;
ALTER TABLE jcm2.building VALIDATE CONSTRAINT chk_building_currentuse;

-- 6.6. Restricciones de Integridad Referencial de Claves Foráneas en SIOSE
ALTER TABLE jcm2.siose_pol ADD CONSTRAINT fk_siose_pol_codiige FOREIGN KEY (codiige)
REFERENCES jcm2.siose_codiige(codiige) NOT VALID;
ALTER TABLE jcm2.siose_pol VALIDATE CONSTRAINT fk_siose_pol_codiige;

ALTER TABLE jcm2.siose_pol ADD CONSTRAINT fk_siose_pol_hilucs FOREIGN KEY (hilucs)
REFERENCES jcm2.siose_hilucs(hilucs) NOT VALID;
ALTER TABLE jcm2.siose_pol VALIDATE CONSTRAINT fk_siose_pol_hilucs;

```

5.10. Resultados

Gracias al nuevo sistema, se pudo obtener trazabilidad sobre los datos de extracción, saneamiento, validación y carga del esquema jcm2.

Capa	SRID Origen	Total Origen (Buffer 500m)	Válidas Origen	Inválidas Origen	Reparadas Éxito	Descartes Escala Micro geometrías	Corruptas / Rotas	Insertadas Destino (jcm2)
building	25830	11.758	11.758	0	0	7	0	11.751
buildingpart	25830	58.908	58.908	0	0	316	0	58.592
cadastralparcel	25830	16.985	16.985	0	0	2	0	16.983
tramovial	4258	8.088	8.088	0	0	0	0	8.088
portalpk	4258	12.908	12.908	0	0	0	0	12.908
tramocurso	4258	422	422	0	0	0	0	422

siose pol	25830	2.307	2.306	1	1	0	0	2.307
ttmm	4258	1	1	0	0	0	0	1

5.10.1. Calidad general

En términos generales la calidad de los datos de origen en jcm1 tienen una alta calidad obteniendo muy pocos errores por corregir. Únicamente 7/11.758 objetos en buildings. Aunque si fue algo más alta en buildingpart con 316 microgeometrías. Aunque el procesamiento fue robusto, en realidad los datos requerían poca intervención

5.10.2. Hallazgos

Evaluando las geometrías descartadas en building generalmente son “astillas” de geometrías, el filtro aplicó muy bien.

Sin embargo, en BuildingPart si hay un error de consistencia, ya que se genera una gran cantidad de buildingparts que bajo mi criterio no deben estar clasificados como buldingparts, ya que no tienen una segmentación funcional o estructural, podría simplificarse y agrupar de mejor manera. Por lo que en principio, los polígonos filtrados están bien. Pero el problema real en la gran mayoría es que son polígonos residuales de incorporar valores estructurales. Por lo que amerita una simplificación manual con criterio. Por ahora lo que va bien es no tener tantos objetos que tiene poca información tanto abstracta como geométrica.



Por otro lado, las Cadastral Parcels hay una “astilla” y un pequeño cuadro entre multiples geometrías, es decir una geometría residual.

Y la geometría invalida del siose era la siguiente.



La cual claramente presenta un anillo interno que linda con el borde del polígono, la corrección fue convertirlo en un polígono sin anillo que rodea el espacio que antes era un anillo.

6. asd